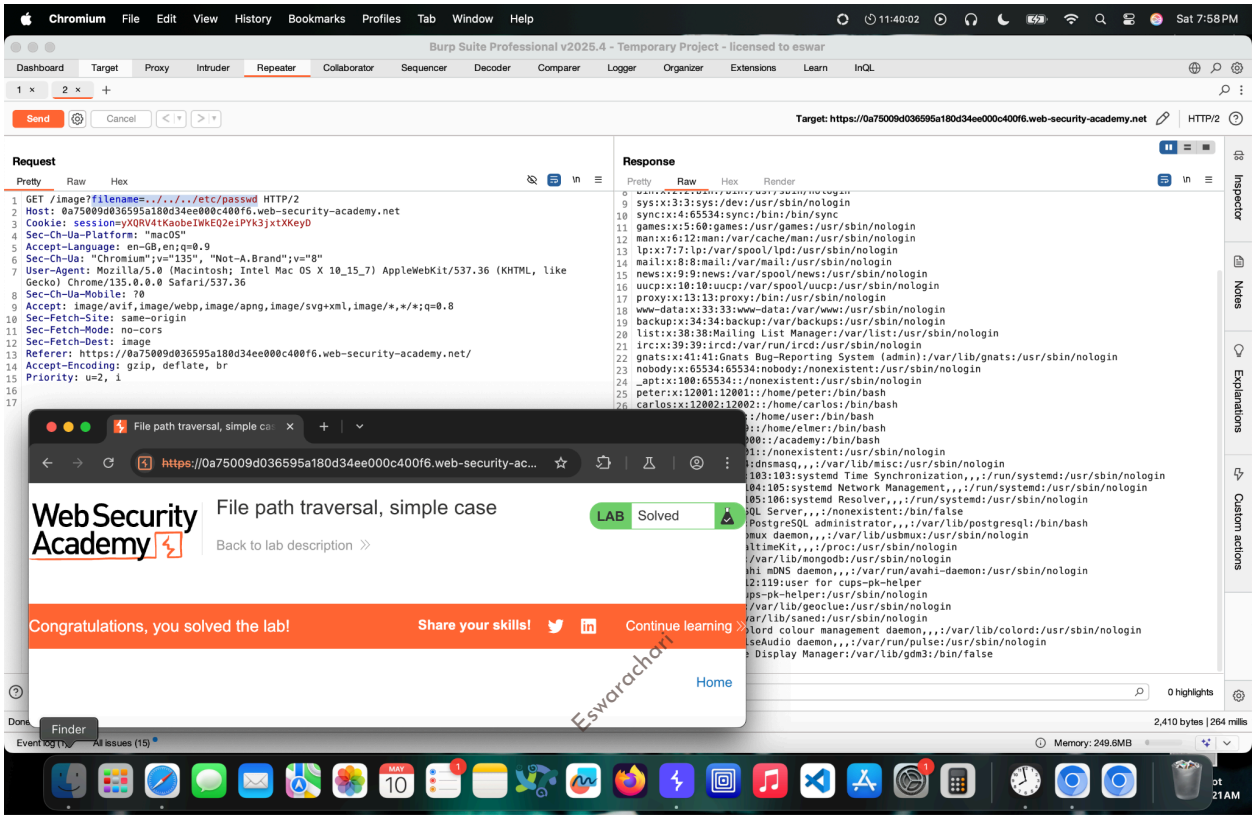
1.Lab Name:File Path Traversal,simple case

Vulnerability Type:File Path Traversal

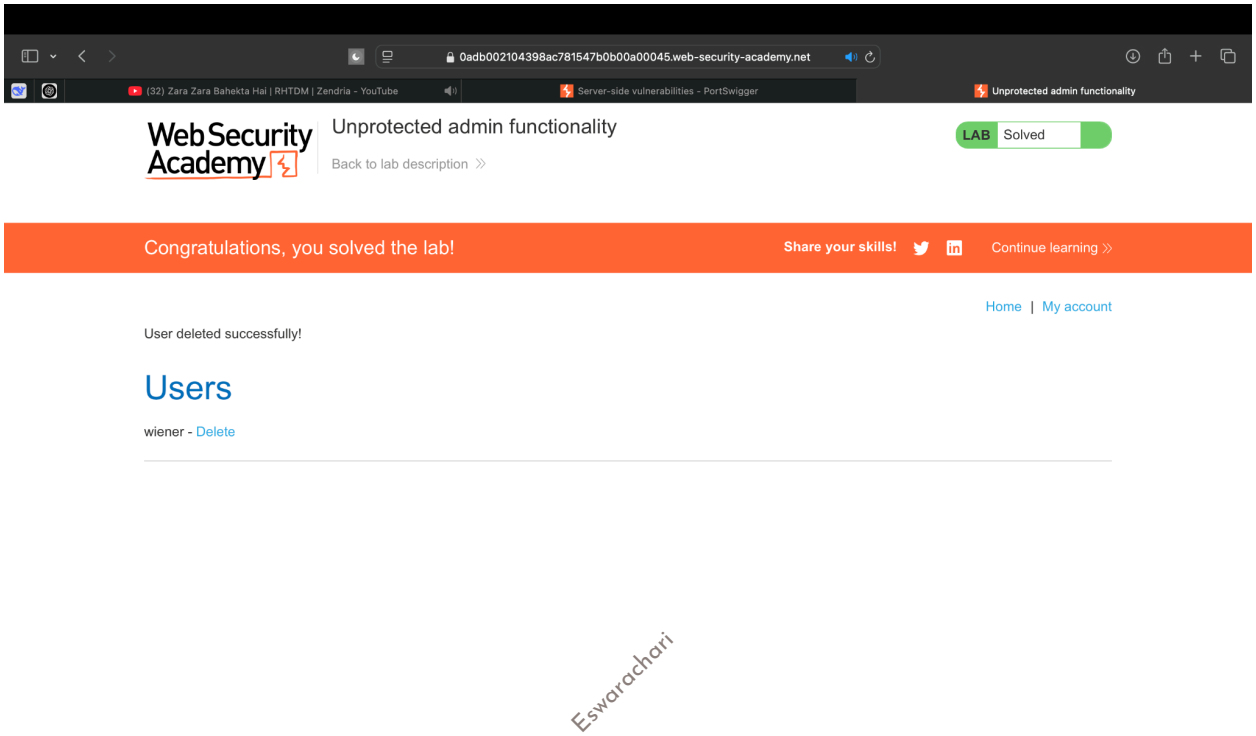


Summary: This lab The lab involved exploiting a file path traversal vulnerability to access sensitive files on the server. I crafted a request to retrieve the /etc/passwd file. This vulnerability can allow attackers to read arbitrary files on the server, potentially exposing sensitive information.

Screenshot Context: The screenshot shows the crafted request to access /etc/passwd and the server's response displaying the file content.

2.Lab Name:Unprotected admin functionality

Vulnerability Type: Unprotected Admin Functionality



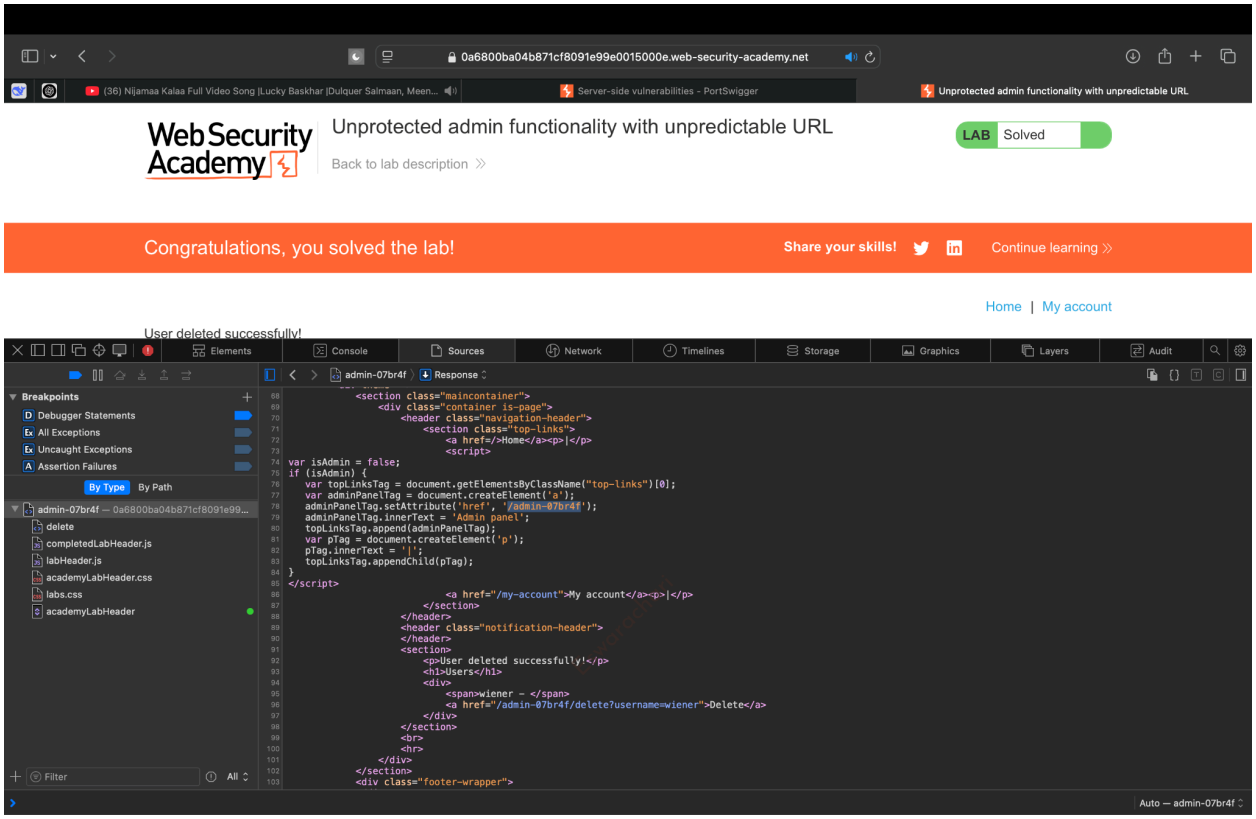
Eswarachari

Summary: This lab focused on accessing an unprotected admin interface. I was able to delete a user through the admin functionality. Unprotected admin functionality can lead to unauthorized modification or deletion of user accounts and other administrative actions.

Screenshot Context: The screenshot shows the admin interface and the action of deleting a user.

3.Lab Name:Unprotected admin functionality with unpredictable URL

Vulnerability Type: Unprotected Admin Functionality

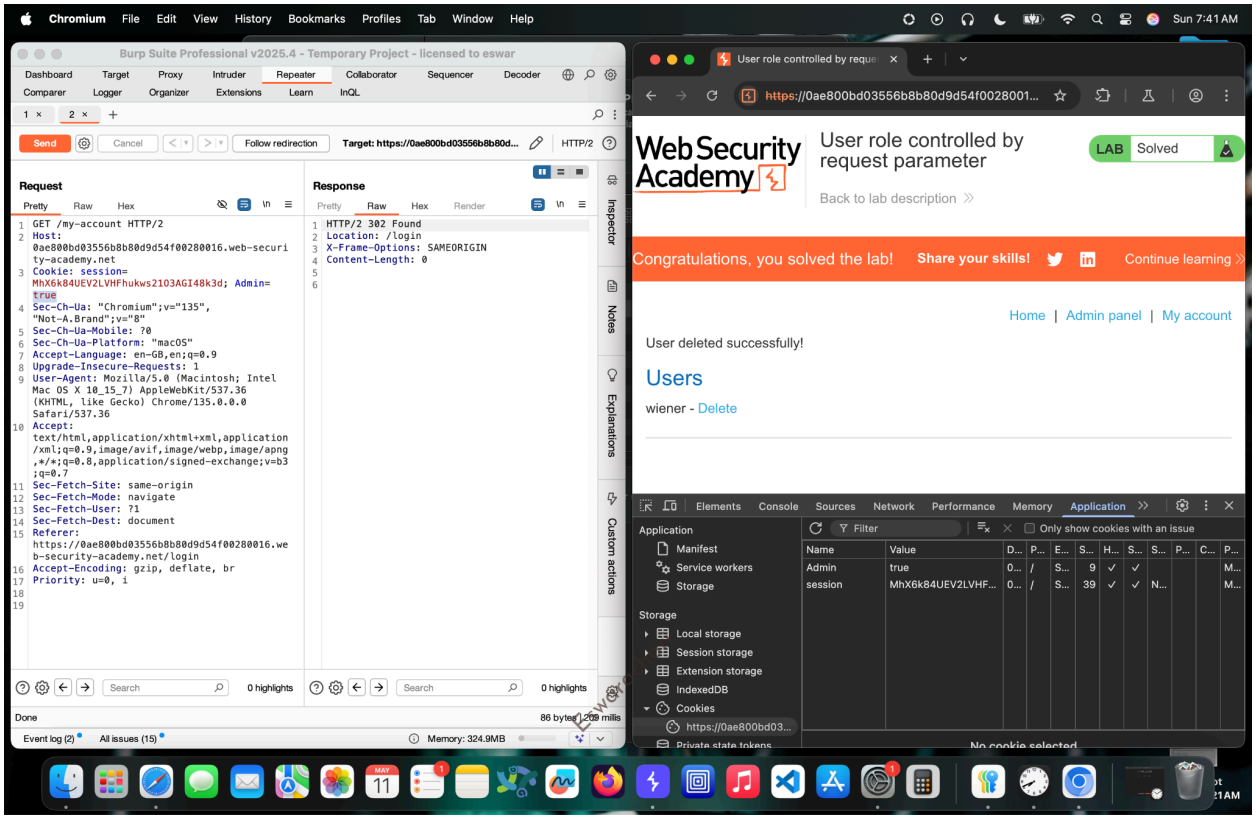


Summary: The lab involved accessing an admin panel with an unpredictable URL. I was able to navigate to the admin panel and observe user deletion. This highlights the risk of relying on URL obscurity for access control, as attackers might discover these URLs.

Screenshot Context: The screenshot shows the admin panel accessed via the unpredictable URL and the user deletion confirmation.

4.Lab Name:User role controlled by request parameter

Vulnerability Type: User Role Controlled by Request Parameter

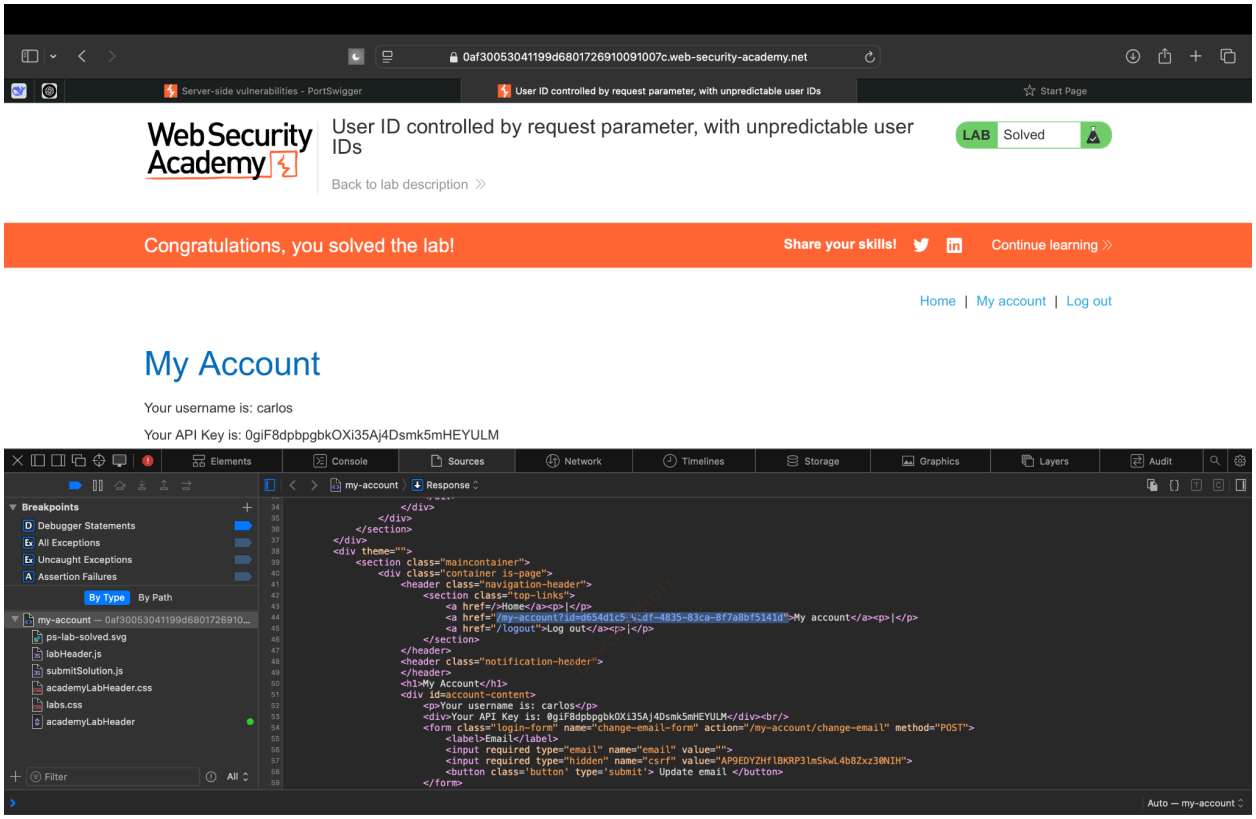


Summary: This lab demonstrated how user roles can be manipulated through request parameters. I modified a parameter to gain admin access. This vulnerability can lead to privilege escalation, where a user gains unauthorized administrative rights.

Screenshot Context: The screenshot shows the modified request with the admin parameter and the resulting admin access.

5.Lab Name:User ID controlled by request parameter, with unpredictable user IDs

Vulnerability Type: User ID Controlled by Request Parameter

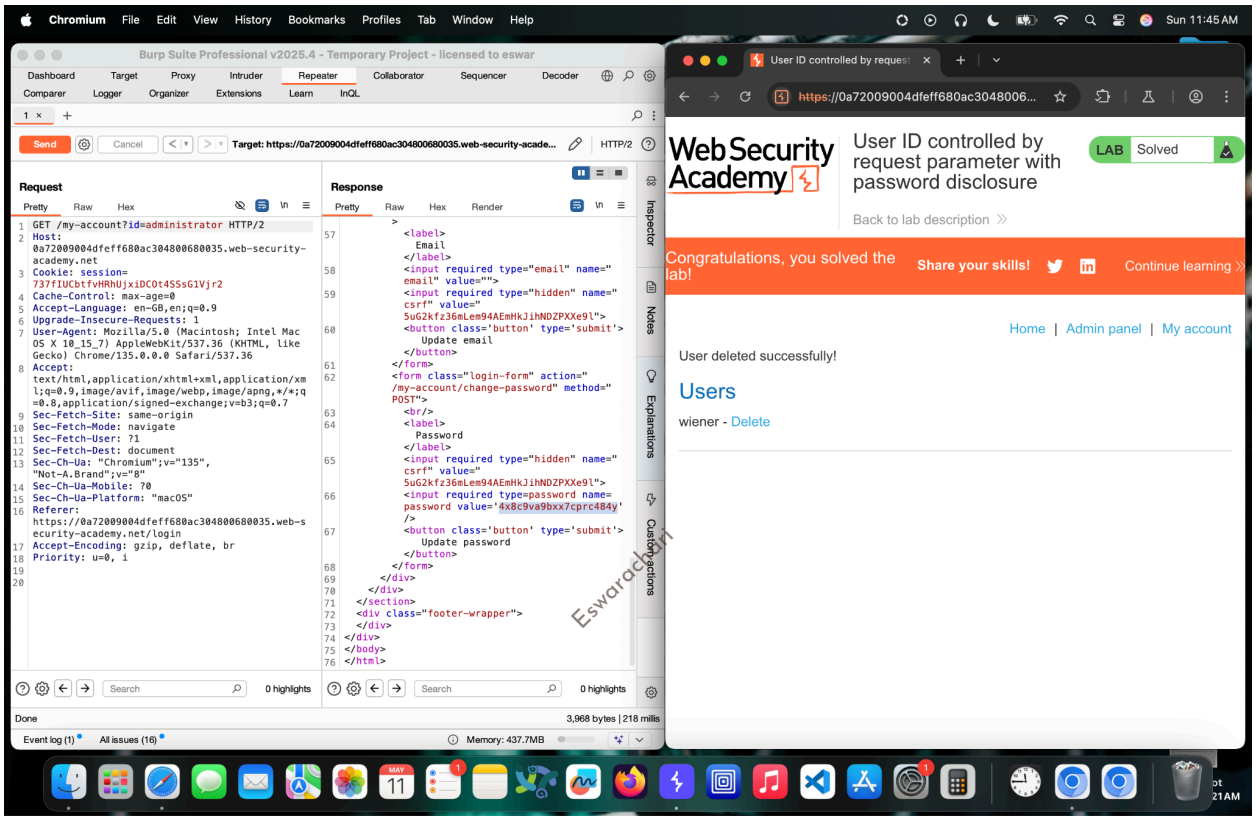


Summary: The lab focused on manipulating user IDs via request parameters to access other user accounts. I was able to view the API key of another user. This vulnerability can lead to information disclosure, where attackers can access sensitive data of other users.

Screenshot Context: The screenshot shows the request with the modified user ID and the response containing the other user's API key.

6.Lab Name:User ID controlled by request parameter with password disclosure

Vulnerability Type: User ID Controlled by Request Parameter with Password Disclosure

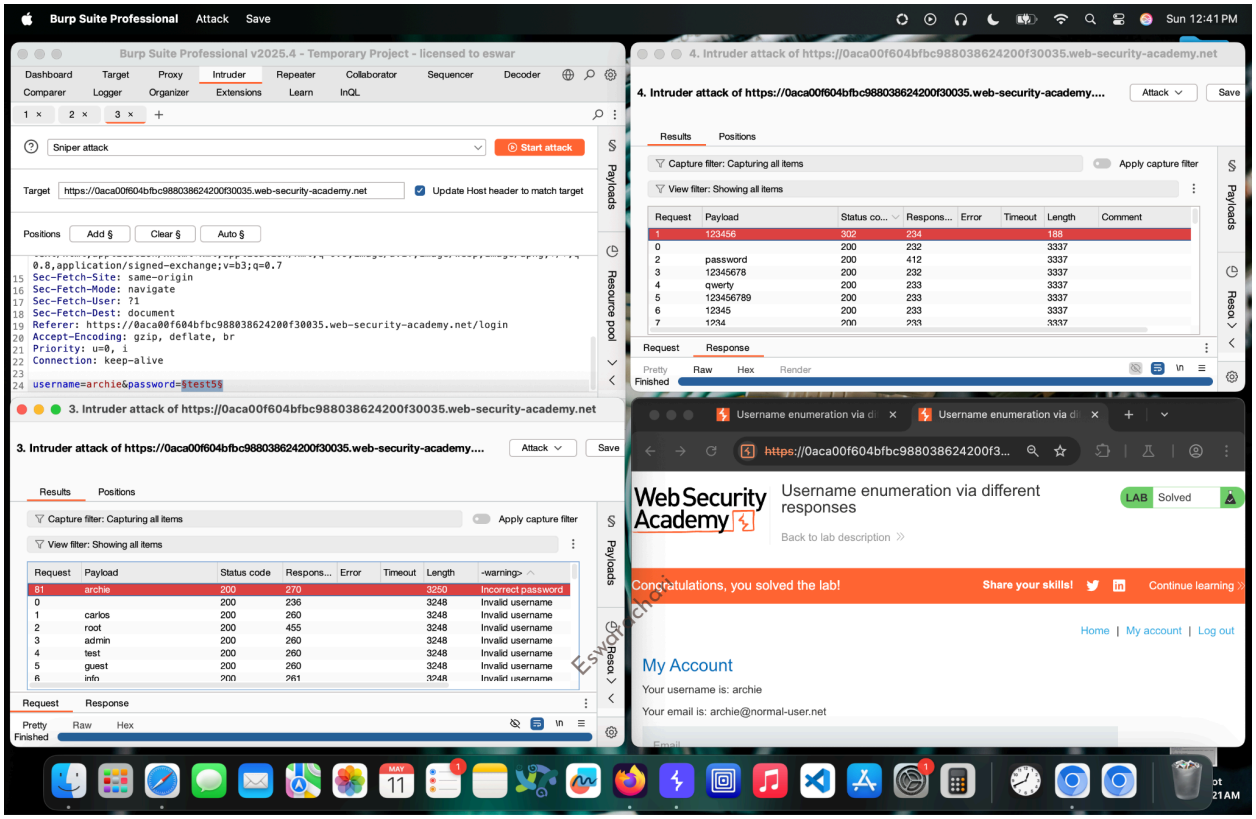


Summary: This lab involved manipulating the user ID parameter to potentially disclose another user's password. By modifying the user ID, it might be possible to retrieve or change passwords of other accounts. This is a critical vulnerability that can lead to complete account takeover.

Screenshot Context: The screenshot shows the manipulated request and the response potentially revealing password information.

7.Lab Name: Username enumeration via different responses

Vulnerability Type: Username Enumeration

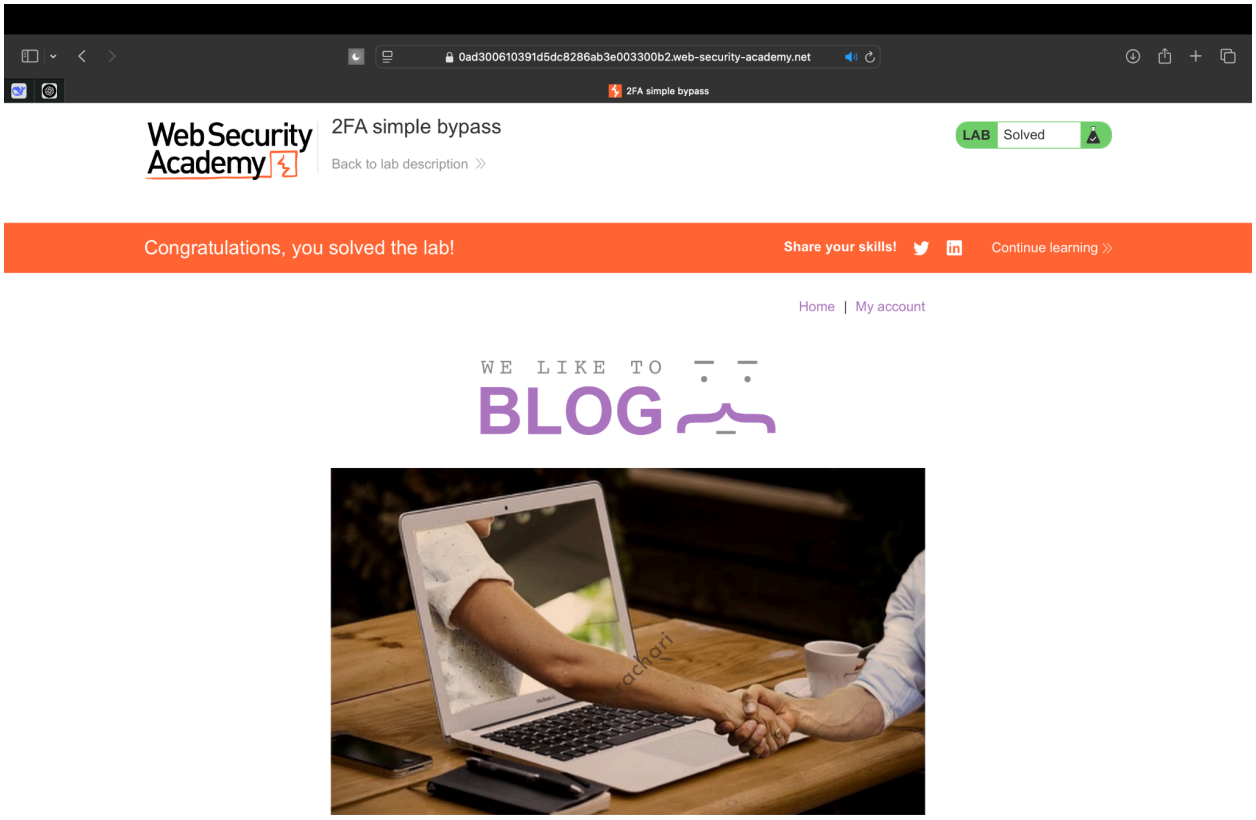


Summary: The lab demonstrated how to enumerate valid usernames by analyzing different server responses. I used intruder to identify a valid username. Username enumeration is often a first step in brute-force attacks to gain unauthorized access.

Screenshot Context: The screenshot shows the intruder attack results, highlighting the different responses for valid and invalid usernames.

8. Lab Name: 2FA simple bypass

Vulnerability Type: 2FA Bypass

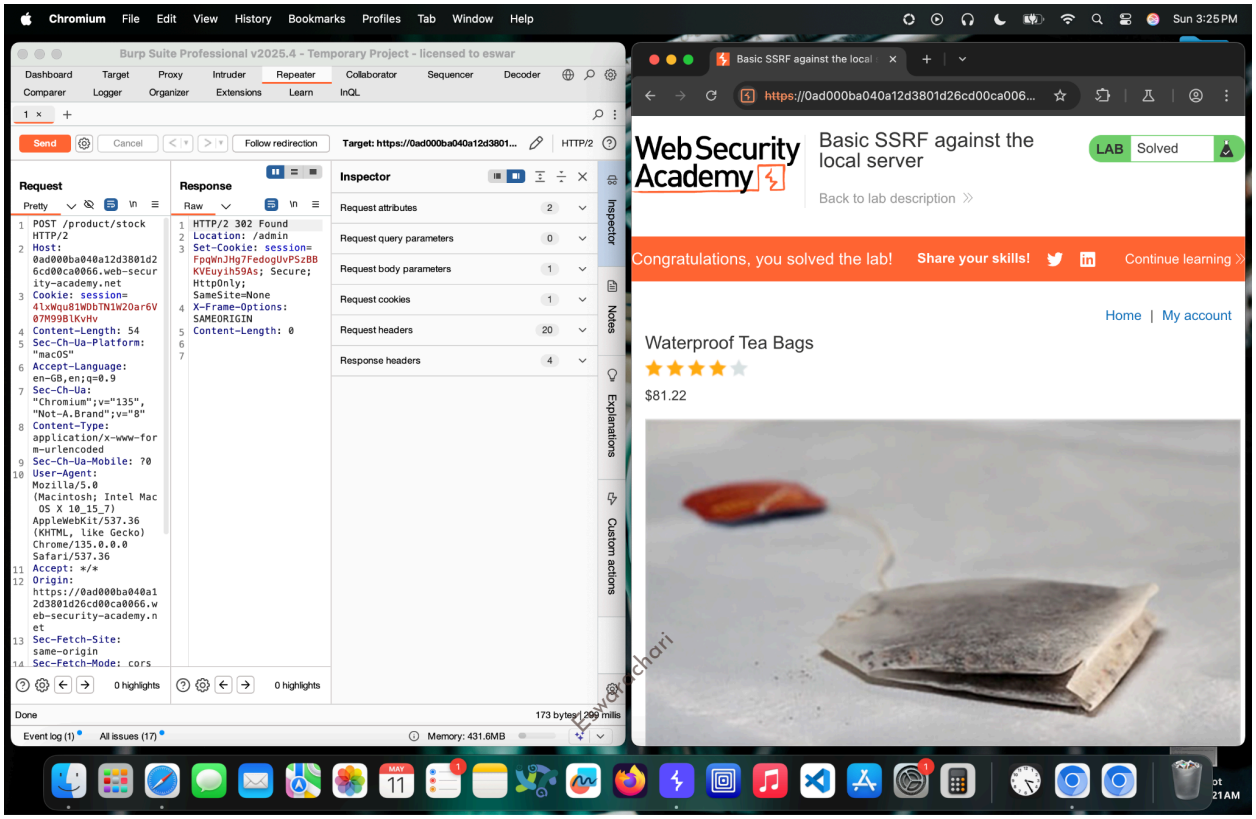


Summary: This lab involved bypassing a two-factor authentication (2FA) mechanism. Successful bypass of 2FA can grant an attacker access to accounts even with the 2FA enabled.

Screenshot Context: This indicates that the two-factor authentication mechanism implemented in this specific lab was successfully bypassed.

9. Lab Name: Basic SSRF against the local server

Vulnerability Type: Server-Side Request Forgery (SSRF)

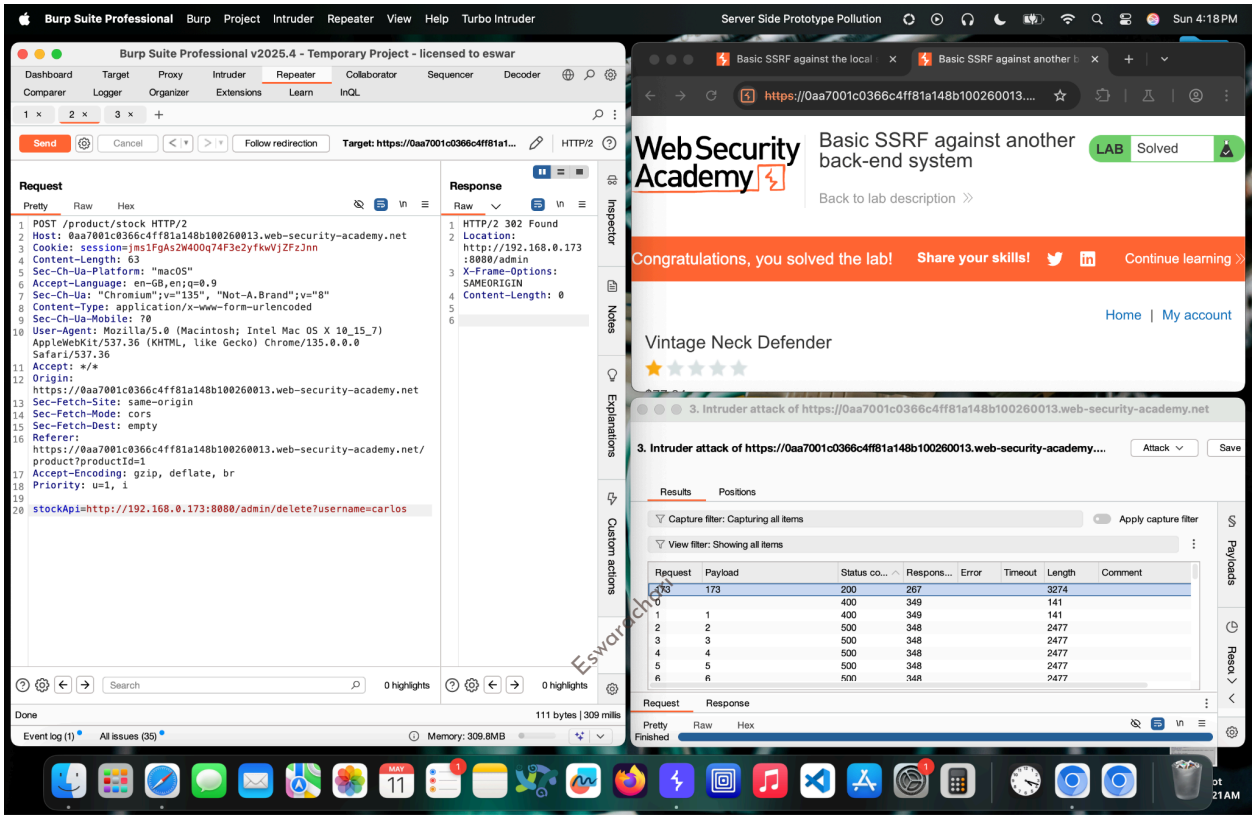


Summary: The lab demonstrated a basic Server-Side Request Forgery (SSRF) vulnerability. I was able to make the server make a request to an internal resource. SSRF vulnerabilities can allow attackers to access internal resources or interact with other systems.

Screenshot Context: The screenshot shows the crafted request that triggers the SSRF vulnerability and the server's response.

10. Lab Name: Basic SSRF against another back-end system

Vulnerability Type: Server-Side Request Forgery (SSRF)

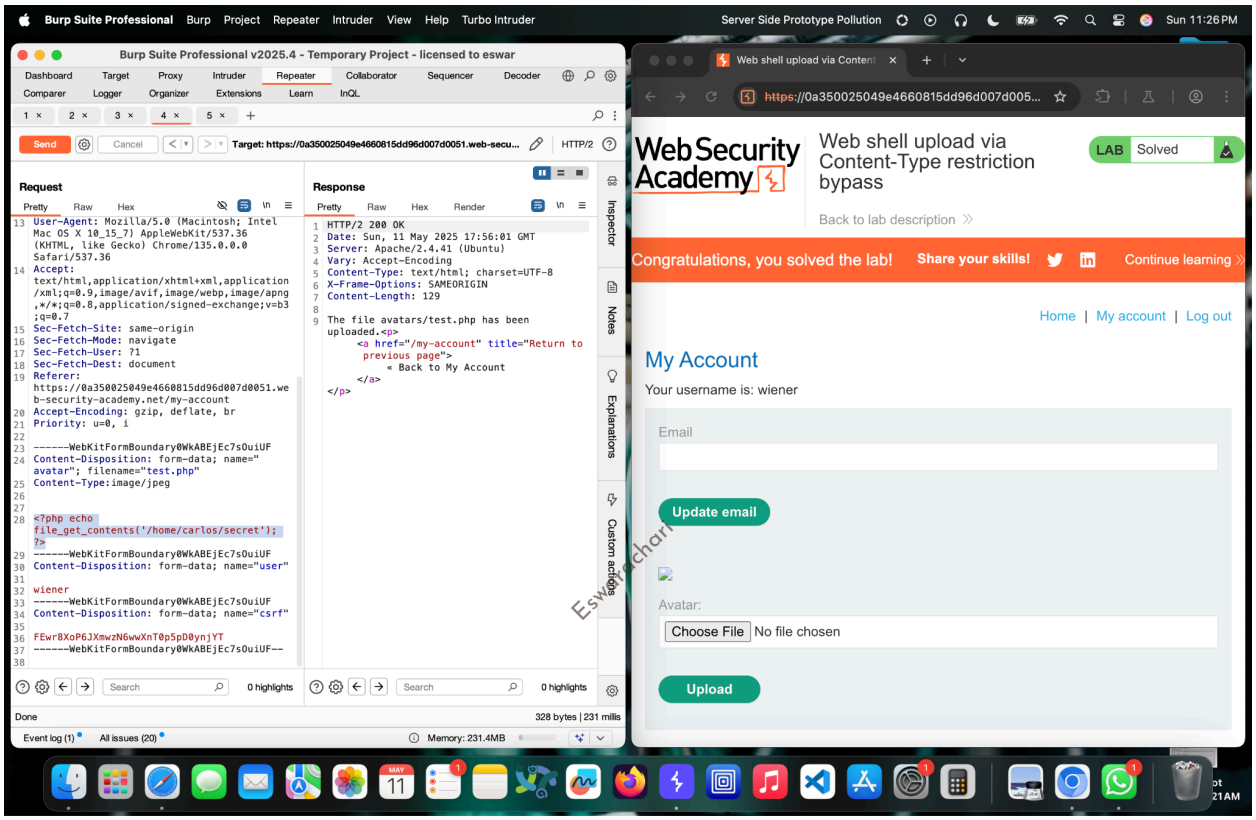


Summary: This lab extended the SSRF concept to target a different backend system. I made the server interact with another internal system. This type of SSRF can be used to pivot to other internal services.

Screenshot Context: The screenshot demonstrating the SSRF attack targeting the backend system and the server's response.

11.Lab Name: Web shell upload via Content-Type restriction bypass (as indicated by the solved banner and page title)

Vulnerability Type: Unrestricted File Upload / Web Shell

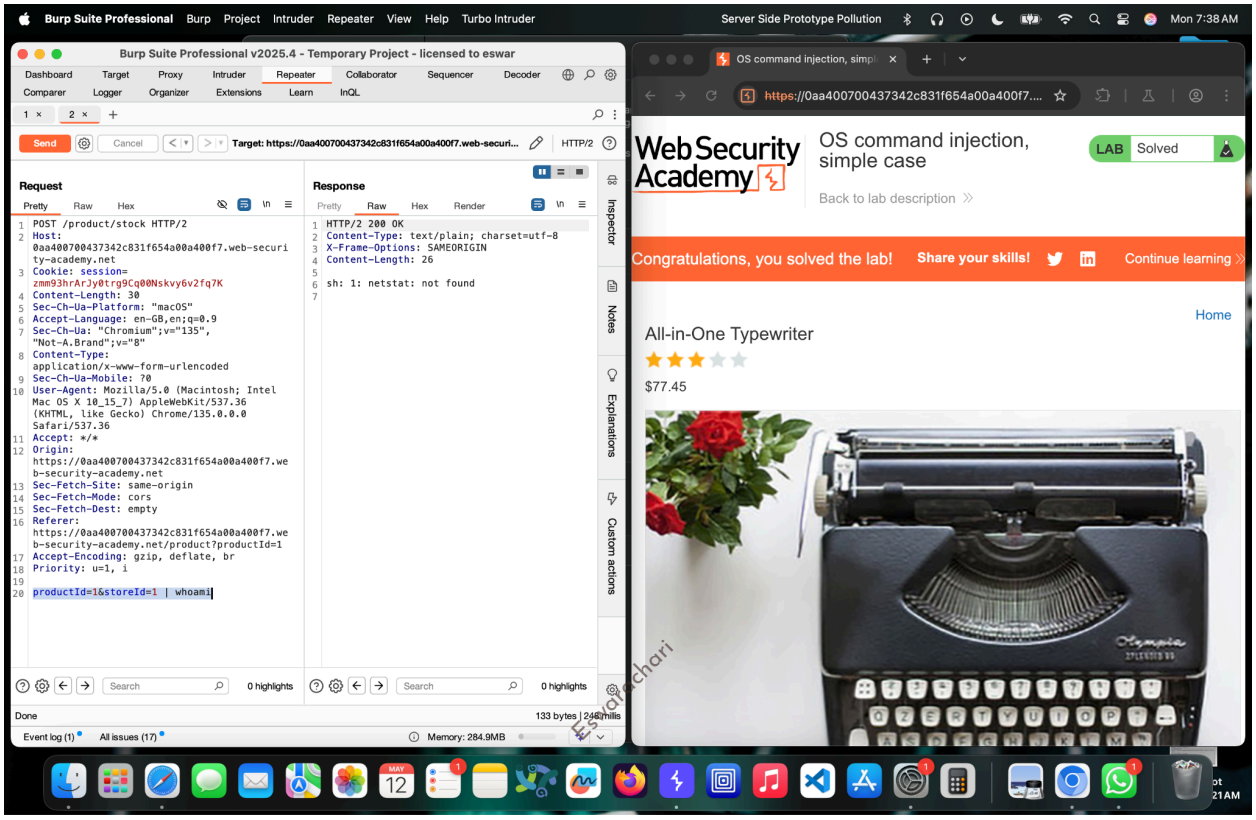


Summary: This lab demonstrates a vulnerability where the application attempts to restrict file uploads based on the Content-Type header, but this restriction can be bypassed. By manipulating the Content-Type in the HTTP request (as seen in the Burp Suite panel), an attacker can upload a malicious file, such as a web shell (test.php), even if it doesn't match the expected content type (like image/jpeg). Once uploaded, the web shell can be accessed and used to execute arbitrary operating system commands on the server, leading to a severe security breach.

Screenshot Context: The screenshot shows a split view. On the left, Burp Suite's Request panel displays a POST request with a manipulated Content-Type header

12.Lab Name: OS command injection, simple case

Vulnerability Type: OS Command Injection

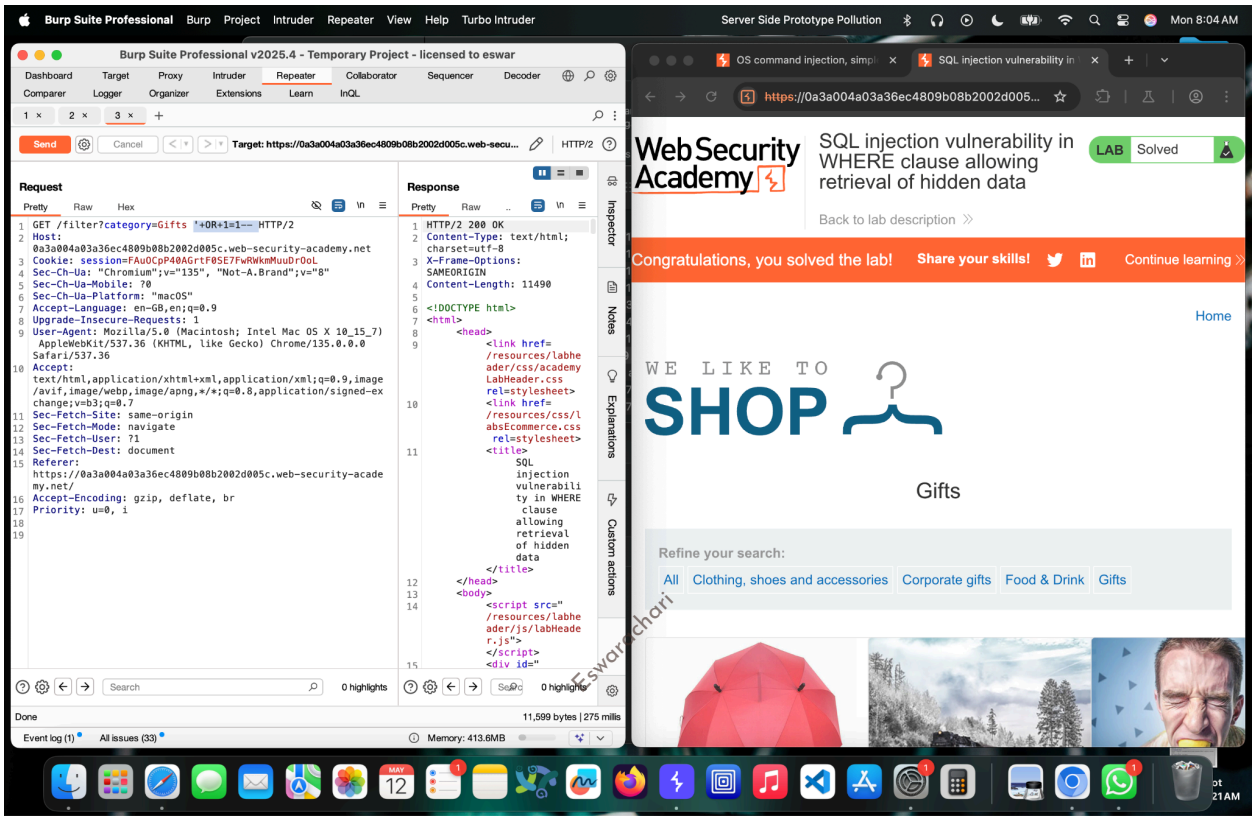


Summary: This lab involves exploiting an OS command injection vulnerability. By manipulating input that is passed to a system command, it's possible to execute arbitrary operating system commands on the server. The successful execution of a command is usually indicated by a change in the application's behavior or output. This type of vulnerability can lead to complete server compromise.

Screenshot Context: The screenshot shows a split view with Burp Suite on the left, displaying a POST request, suggesting the manipulation of input parameters. On the right, the application page displays product details, indicating a change in the application's output as a result of the successful command injection.

13.Lab Name: SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

Vulnerability Type: SQL Injection

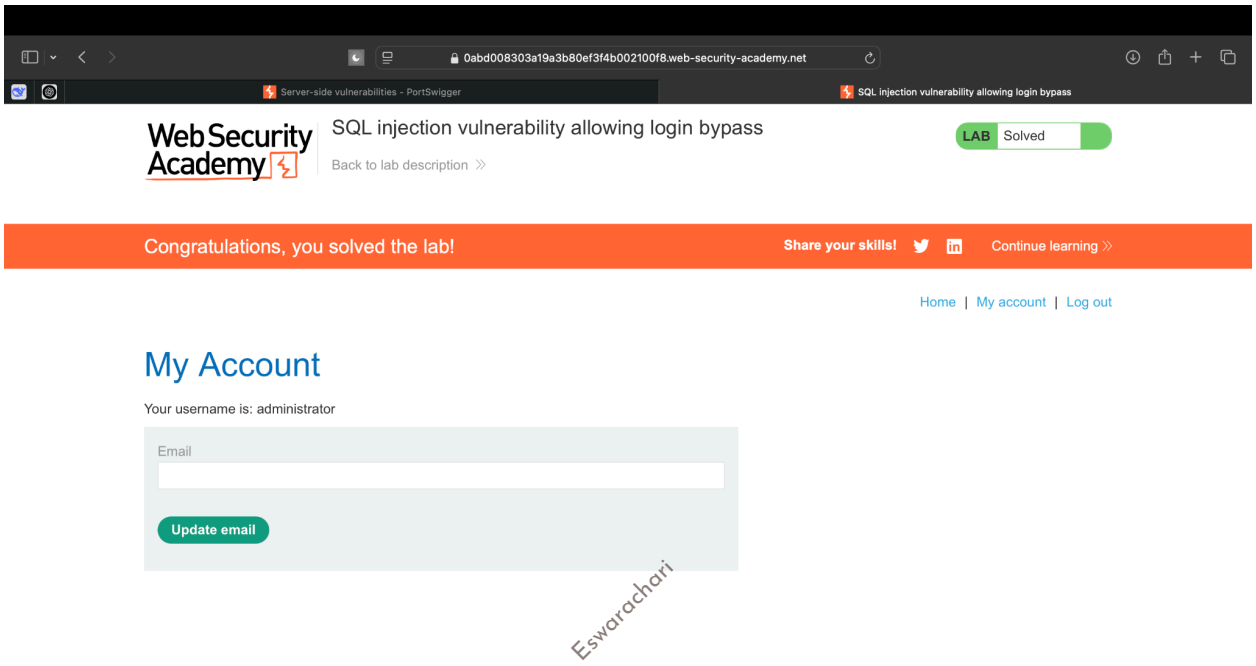


Summary: This lab demonstrates a SQL injection vulnerability within the `WHERE` clause of a SQL query. By injecting malicious SQL code into a user-controlled input field (likely related to filtering or searching products), it's possible to manipulate the query to bypass intended logic and retrieve hidden data that would not normally be accessible. This could involve viewing sensitive product details, user information, or other confidential data.

Screenshot Context: The screenshot shows a split view with Burp Suite on the left, showing a `GET` request to a `/filter`endpoint, implying the injection of a SQL payload within the filtering parameters. On the right, the application displays a product listing page, indicating the successful retrieval of data, likely including hidden items, due to the SQL injection.

14.Lab Name: SQL injection vulnerability allowing login bypass

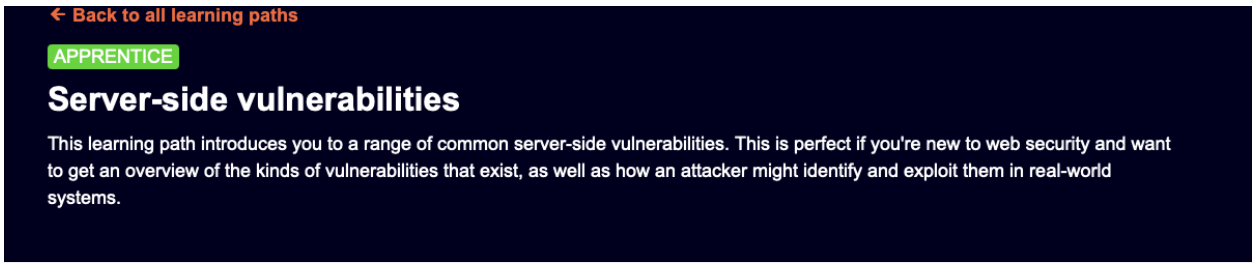
Vulnerability Type: SQL Injection



Summary: This lab demonstrates a critical SQL injection vulnerability that allows an attacker to bypass the login authentication process. By injecting malicious SQL code into the username or password fields, it's possible to manipulate the login query in a way that always evaluates to true, granting unauthorized access to the application. This allows attackers to bypass authentication and gain access to user accounts, often with elevated privileges.

Screenshot Context: The screenshot shows the application interface displaying a "My Account" page with the username **"administrator,"** signifying successful login bypass via SQL injection. The focus is on the authenticated state rather than the login form itself.

Server-side vulnerabilities: Key Topics Explored:



Contents

Resume now: Subverting application logic

51 of 52

RESUME LEARNING →

Path traversal 3 of 3

Path traversal (also known as directory traversal) vulnerabilities enable an attacker to interact with arbitrary files on the server, giving them access to sensitive data. If they can also write to these files, they can potentially modify application data or behavior, ultimately taking full control of the server.

- What is path traversal?
- Reading arbitrary files via path traversal
- Lab: File path traversal, simple case APPRENTICE

Eswarachari

Access control 12 of 12

Access controls are designed to prevent users from interacting with data or functionality for which they don't have the relevant permissions. Due to the obvious security impact, broken access controls are critical bugs in their own right. They often also provide access to more attack surface, which could contain additional vulnerabilities.

- What is access control?
- Vertical privilege escalation
- Unprotected functionality
- Lab: Unprotected admin functionality APPRENTICE
- Unprotected functionality - Continued
- Lab: Unprotected admin functionality with unpredictable URL APPRENTICE
- Parameter-based access control methods
- Lab: User role controlled by request parameter APPRENTICE
- Horizontal privilege escalation
- Lab: User ID controlled by request parameter, with unpredictable user IDs APPRENTICE
- Horizontal to vertical privilege escalation
- Lab: User ID controlled by request parameter with password disclosure APPRENTICE

Authentication 10 of 10

Authentication is the process of checking that a user really is who they claim to be. For example, a login form where you enter your username and password is one form of authentication mechanism. Flawed authentication mechanisms may allow an attacker to guess valid sets of credentials by automating thousands of login attempts using specialist tools like Burp Intruder.

- Authentication vulnerabilities
- What is the difference between authentication and authorization?
- Brute-force attacks
- Brute-forcing usernames
- Brute-forcing passwords
- Brute-forcing passwords - Continued
- Username enumeration
- Lab: Username enumeration via different responses APPRENTICE
- Bypassing two-factor authentication
- Lab: 2FA simple bypass APPRENTICE

Server-side request forgery (SSRF) 6 of 6

SSRF vulnerabilities enable an attacker to trigger malicious server-to-server requests to unintended URLs. As the server issuing the request is likely to have a strong trust relationship with other systems on the network, the attacker can potentially abuse this behavior to access data, functionality, and services that are not meant to be exposed to external users.

- What is SSRF?
- SSRF attacks against the server
- SSRF attacks against the server - Continued
- Lab: Basic SSRF against the local server APPRENTICE
- SSRF attacks against other back-end systems
- Lab: Basic SSRF against another back-end system APPRENTICE

Eswarachari

File upload vulnerabilities 9 of 9

Any functionality that enables users to upload files to the server's filesystem are inherently dangerous. Failing to enforce proper restrictions on the files that users are allowed to upload can potentially enable an attacker to run arbitrary system commands, giving them full control over the server.

- What are file upload vulnerabilities?
- How do file upload vulnerabilities arise?
- Exploiting unrestricted file uploads to deploy a web shell
- Lab: Remote code execution via web shell upload APPRENTICE
- Exploiting flawed validation of file uploads
- Flawed file type validation
- Flawed file type validation - Continued
- Flawed file type validation - Continued
- Lab: Web shell upload via Content-Type restriction bypass APPRENTICE

OS command injection 5 of 5

Command injection vulnerabilities enable an attacker to execute arbitrary operating system (OS) commands on the server. This gives them full control over the server, compromising the application and all of its data.

- What is OS command injection?
- Useful commands
- Injecting OS commands
- Injecting OS commands - Continued
- Lab: OS command injection, simple case APPRENTICE

SQL injection 6 of 7

SQL injection is a classic vulnerability that is responsible for many high-profile data breaches. It enables an attacker to interfere with the queries the application issues to its database, potentially returning sensitive data from arbitrary tables within the database.

- What is SQL injection (SQLi)?
- How to detect SQL injection vulnerabilities
- Retrieving hidden data
- Retrieving hidden data - Continued
- Lab: SQL injection vulnerability in WHERE clause allowing retrieval of hidden data APPRENTICE
- **Subverting application logic** Resume →
- Lab: SQL injection vulnerability allowing login bypass APPRENTICE

Eswaracki